

A minor mishap in the pipeline..

Earlier we had introduced Qwen before bge-m3 as phase 2, to irradicate query formatting sensitivity. What we didnt notice was that we had introduced Qwen twice in the pipeline– one for Pre-Retrieval Extraction and the other for Post-Retrieval Generation. **This could lead to hardware load and latency across the dual-SLM architecture by running two Qwen calls as it could impact CPU/MPS memory limits.**

We physically cannot combine Pre-Retrieval Extraction and Post-Retrieval Generation into a single LLM inference call without breaking the RAG architecture.

In a cascaded pipeline, these two calls are separated by a hard causal boundary: the database search.

- **Call 1 (Extraction)** creates the search key.
- **ChromaDB & DeBERTa** use that key to find and verify the facts.
- **Call 2 (Generation)** requires those verified facts to format the final Telugu JSON.

We cannot generate the answer before we retrieve the facts, and we cannot retrieve the facts until we extract the intent. **However**, we can drastically reduce the compute overhead and latency of running two Qwen calls by implementing **Asymmetric Execution** and **Cache State Preservation**.

Here is how we can engineer the pipeline to minimize the dual-call penalty:

1. The NLI Short-Circuit (Conditional Generation)

We only actually need the second Qwen call when the DeBERTa gate outputs **ANSWER**.

- If the gate outputs **CLARIFY**, skip the second SLM call entirely. Have the Streamlit UI immediately output a hardcoded template: *"I need more information. What is the current [missing parameter]?"*
- If the gate outputs **ABSTAIN**, skip the second SLM call. Hardcode the UI to throw a critical red flag: *"Warning: This action contradicts safe aquaculture practices."*
- This means your system only pays the "two-call tax" when it has a perfect, verified diagnosis.

2. State Preservation in llama.cpp (KV Caching)

The biggest latency hit in a dual-SLM setup is loading the model weights into RAM and processing the system prompt twice. We must optimize the llama.cpp Python bindings:

- **Load Once:** Initialize the Qwen model globally in the Streamlit session state, not inside the function call.
- **Prompt Caching:** Enable `prompt_cache` in the llama.cpp configuration. The model will cache the tokenized state of its base instructions, meaning the second call only has to compute the newly retrieved chunks, cutting the Time-to-First-Token (TTFT) nearly in half.

3. Asymmetric Model Downsizing (The Ultimate Fix)

Qwen-2.5-1.5B is required for the final generation because it has to translate English aquaculture facts into strict Telugu JSON. However, it is complete overkill for flattening a messy sentence into a declarative hypothesis.

- Swap the Stage 2 Extractor to a micro-model like **Qwen-2.5-0.5B** (or even a tiny fine-tuned ONNX intent classifier).
- Keep **Qwen-2.5-1.5B** exclusively for the Stage 5 Generation.
- Running a 0.5B model followed by a 1.5B model requires significantly less CPU memory bandwidth than running the 1.5B model twice, easily keeping us under our 3.0-second latency budget.

Updated Pipeline:

Phase 0: RLAIIF Data Engine & Training (Offline/Pre-Compute)

- **Data Synthesis:** The automated RLAIIF data engine script (`src/generate_rlaif_data.py`) synthesizes cross-lingual NLI training triplets (Entailment, Neutral, Contradiction). It generates "messy farmer paragraphs" paired with clean declarative hypotheses for edge-case evaluation.
- **The Chunk Fix:** The dataset strict-maps to the 50-word blocks from `chunks.json` to bypass the cross-encoder token-truncation loophole. **[Praisys currently working on it]**
- **Biological Filter:** The `verify_aquaculture_invariants` Python assertion suite automatically filters out biologically impossible values (e.g., pH bounds outside [6.0, 9.5]) before dataset admission. **[a new thing; also needs to be done by Praisys]**

Phase 1: Knowledge Base Ingestion & Indexing **[Completed]**

- **Corpus Indexing:** ICAR-CIBA manuals are parsed into `chunks.json`, tagged with metadata, and embedded into a local ChromaDB instance via BGE-M3 to support exact numerical and semantic matching.

Phase 2: Core Diagnostic Routing Engine (Extraction, Retrieval & Verification) *[to start from here after Phase 0 work is completed]*

- **Step A: Asymmetric Intent Extraction:** The messy, colloquial farmer input is fed to a fast, lightweight SLM extraction prompt (Qwen-2.5-0.5B). The SLM instantly flattens the conversational input into a clean, declarative diagnostic hypothesis.
- **Step B: Hybrid Bi-Encoder Retrieval:** ChromaDB uses the BGE-M3 retriever to map the clean declarative hypothesis against the database, pulling the top-K relevant English chunks containing specific treatment protocols.
- **Step C: Cross-Encoder Decision Gate (The Short-Circuit):** The DeBERTa cross-encoder evaluates the extracted hypothesis against the retrieved chunks to trigger the three-way state machine:
 - **ABSTAIN (P(Contradiction) > 0.30):** Biologically unsafe input. Halt pipeline, skip Phase 3, and issue a hardcoded biological safety warning.
 - **CLARIFY (P(Neutral) > 0.40):** Missing information. Halt pipeline, skip Phase 3, and trigger the UI to prompt the user (e.g., "Please provide current pH...").
 - **ANSWER (P(Entailment) >= 0.75):** Perfect diagnostic match. Advance to Phase 3.

Phase 3: Guardrailed Generation & Conversational UI

- **Offline Audio Integration:** `whisper.cpp` runs within the local Streamlit browser UI (`localhost:8501`) to handle real-time colloquial Telugu transcription.
- **KV Caching Execution:** The quantized Qwen-2.5-1.5B model is called only if Phase 2 outputs ANSWER. `llama.cpp` prompt-caching keeps the end-to-end Time-to-First-Token (TTFT) under 3.0 seconds on a standard CPU.
- **Grammar Masking:** A `constraints.gbnf` context-free grammar file locks the model output into strict JSON structure compliance, ensuring fields for `evidence_chunk_id`, `parameter_identified`, and `recommended_action_telugu` without hallucinations.

Detailed Task Distribution

M1: Khushi (Lead Architect: Pipeline Logic & Routing)

- **Extraction Prompting (Phase 2A):** Engineer the strict zero-shot system prompt forcing the lightweight Qwen-2.5-0.5B model to compress messy paragraphs into flat, declarative hypotheses.

- **NLI Gate Integration (Phase 2C):** Update `src/decision_router.py` to ensure DeBERTa mathematically evaluates the newly extracted hypothesis against the retrieved chunks.
- **Threshold Enforcement (Phase 2C):** Hardcode the deterministic routing thresholds: ANSWER (≥ 0.75), CLARIFY (> 0.40), and ABSTAIN (> 0.30).
- **Empirical Ablation:** Execute the `run_ablation.py` sweeps using the `chunks.json` bypass to map the high-resolution precision, recall, and safety-coverage curves across the consolidated Phase 2 engine.

M2: Meghana (Development: Edge UI, Inference & Formatting)

- **Audio Transcription (Phase 3):** Integrate the `whisper.cpp` bindings into the Streamlit frontend to transcribe spoken Telugu audio directly into the pipeline.
- **Asymmetric SLM Wiring (Phase 2A & 3):** Configure the `llama.cpp` bindings to load two models: the highly compressed Qwen-2.5-0.5B for immediate extraction, and Qwen-2.5-1.5B utilizing tokenized KV prompt-caching for the final generation.
- **Short-Circuit UI (Phase 2C):** Build the Streamlit logic to intercept the CLARIFY and ABSTAIN states, serving hardcoded follow-up questions and safety warnings without triggering the 1.5B model.
- **JSON Enforcement (Phase 3):** Write and apply the `constraints.gbnf` context-free grammar file to lock the Phase 3 generation into strict JSON format.

M3: Jyothish (Documentation, Profiling & Diagrams)

- **Latency Benchmarking:** Profile the hardware load for the asymmetric dual-SLM setup. Log the CPU/MPS memory footprints and ensure the end-to-end TTFT stays under the 3.0-second threshold.
- **Architecture Diagramming:** Redraw the system flowchart to depict the consolidated Phase 2 (combining extraction, retrieval, and DeBERTa gating) and explicitly map the CLARIFY/ABSTAIN bypass arrows skipping Phase 3.
- **Literature Curation:** Add Zotero citations defending "Zero-Shot Intent Extraction" and "Query Rewriting in RAG" for the methodology sections of the IEEE manuscript.

🌟 M4: Praisyl (Data Engine & Edge-Case Synthesis) / Phase 0: RLAIIF Data Engine & Verification Tasks

1. Implement the Chunk Truncation Fix (`src/generate_rlaif_data.py`)

- Update the synthetic dataset generation script to stop dumping full, unchunked PDF text into the dataset's `evidence` field.
- Strictly map the dataset to use `evidence_chunk_id` so it directly references the 50-word English chunks generated in `chunks.json`.
- This fix is a hard requirement to bypass the 512-token limit that currently causes the DeBERTa cross-encoder to stall and output false "Neutral" states.

2. Build the Biological Invariant Filter

- Develop the `verify_aquaculture_invariants` Python assertion suite to sanitize the synthetic training triplets before they are admitted into the final dataset.
- Enforce physical and chemical aquaculture constraints to filter out hallucinated data (e.g., programmatically reject any generated data where pH bounds fall outside [6.0, 9.5] or where dissolved oxygen levels are biologically impossible).

3. Synthesize Messy Edge-Case Data

- Configure the Teacher LLM (Llama-3.3-70B) prompts to generate realistic, messy "farmer paragraphs" (e.g., conversational, dialectal symptom descriptions) rather than neat academic queries.
- Ensure these messy queries are accurately paired with their cleanly extracted declarative hypotheses and the expected routing labels (Entailment, Neutral, Contradiction).

4. Final Repository Handoff

- Push the corrected `generate_rlaif_data.py` script and the invariant filter suite to the repository's master branch.
- Generate and push the final, sanitized `rlaif_dataset.json` to the central repository to officially close Phase 0 and fully unblock the Phase 2 extraction and routing logic.

Execution Task Flow

Step 1: Data Synthesis & Baseline Establishment (M4 - Praisyl)

- **Action:** Detailed task distribution given above.
- **Trigger:** Once the **Chunk Truncation Fix** (mapping evidence to `evidence_chunk_id`) and the **Biological Invariant Filter** inside `src/generate_rlaif_data.py` are successfully implemented, and pushes the sanitized dataset and updated code to the GitHub master branch, her part of Phase 0 is officially complete. This instantly unblocks Khushi to begin ablation.

Step 2: Core Diagnostic Routing Implementation (M1 - Khushi)

- **Action:** Khushi injects the zero-shot extraction prompt at the start of the routing pipeline. She updates `decision_router.py` to evaluate the flattened hypotheses and locks in the 0.75/0.40/0.30 probability gates. She then runs the ablation sweep on Praisyl's newly pushed dataset.
- **Trigger:** Khushi pushes the finalized routing logic and extraction prompt. This locks down the pipeline's intelligence and unblocks Meghana's UI integration.

Step 3: Edge Inference & UI Assembly (M2 - Meghana)

- **Action:** Meghana wires `llama.cpp` to execute the Qwen-2.5-0.5B extraction prompt first. She builds the Streamlit short-circuits to handle DeBERTa's CLARIFY and ABSTAIN outputs. For ANSWER states, she enables KV caching for the Qwen-2.5-1.5B model and enforces the `constraints.gbnf` output mask. She integrates `whisper.cpp` into the frontend.
- **Trigger:** The end-to-end local Streamlit application successfully processes a spoken Telugu query, flattens it, retrieves data, routes it, and generates the JSON output without crashing.

Step 4: Validation & Manuscript Preparation (M3 - Jyothish)

- **Action:** Jyothish stress-tests Meghana's completed Streamlit app, recording the Time-to-First-Token (TTFT) and memory overhead. He updates the architectural diagrams to reflect the newly consolidated Phase 2 short-circuiting and organizes the bibliography for the final draft.